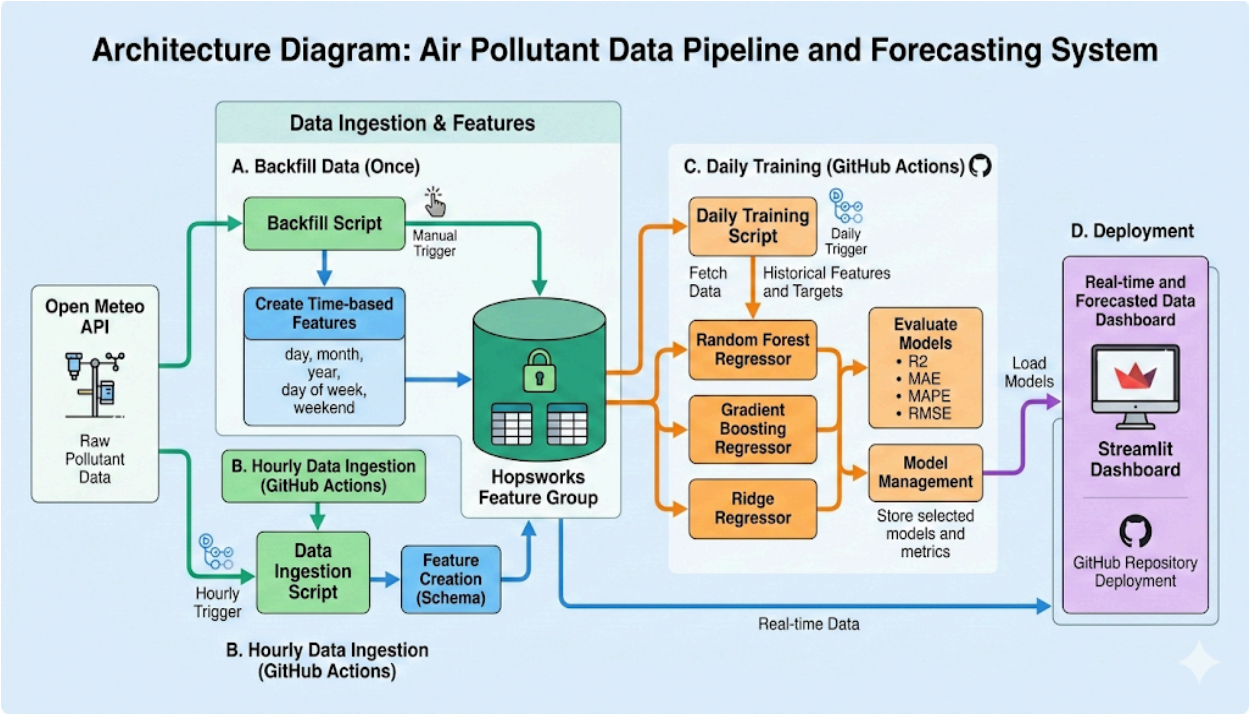
AQI Predictor

Introduction

- **Project Overview**: Building a 100% serverless AQI prediction service that forecasts the air quality of Karachi for the next 3 days.
- **Core Technology Stack**:
  - Open Meteo API for data collection.
  - Numpy, Pandas for feature engineering.
  - Matplotlib and Seaborn for exploratory data analysis.
  - Hopsworks feature store and model registry for storing data and models.
  - Scikit-Learn library for model training and evaluation.
  - SHAP analysis for feature importance.
  - Streamlit for a web dashboard.

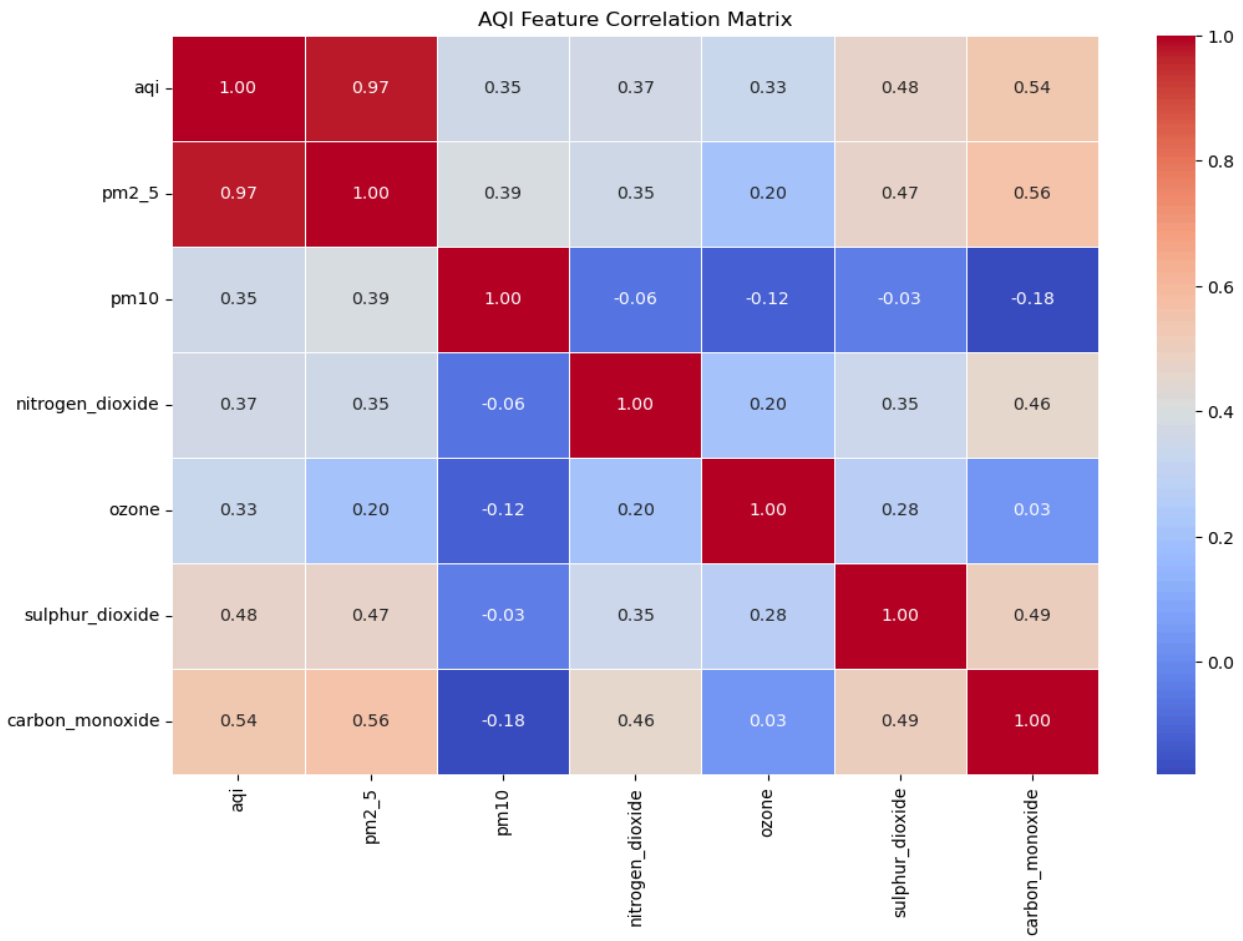


Exploratory Data Analysis

Features extracted from Open Meteo's air-quality API:

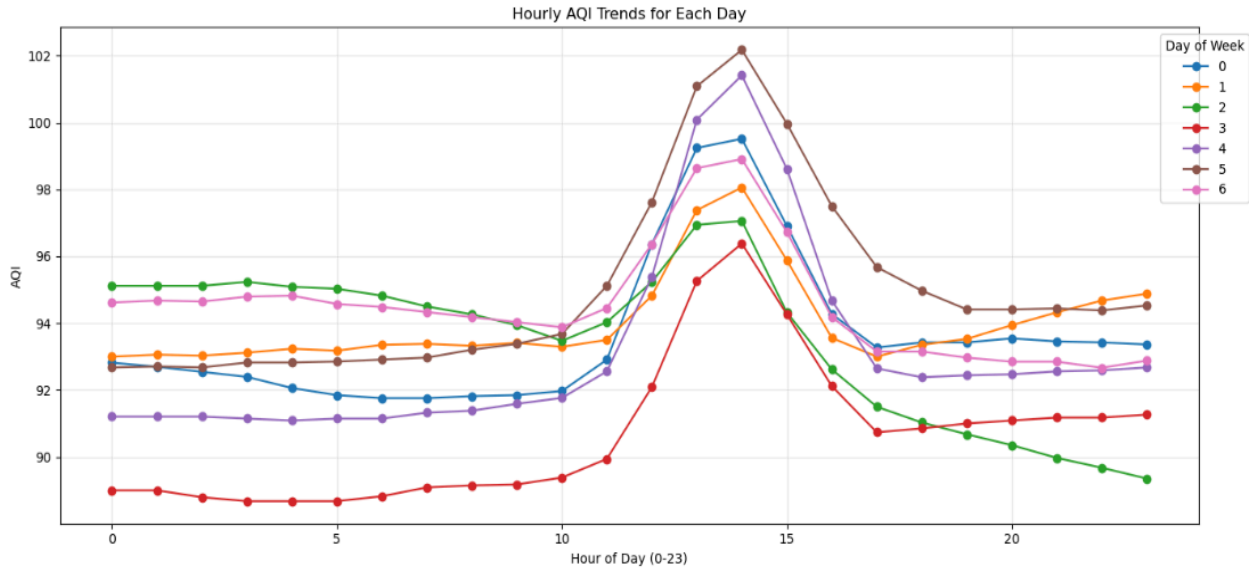
- Time
- AQI
- PM2.5
- PM10
- Nitrogen Dioxide (NO2)
- Ozone (O3)
- Sulphur Dioxide (SO2)
- Carbon Monoxide (CO)

Correlation Matrix:



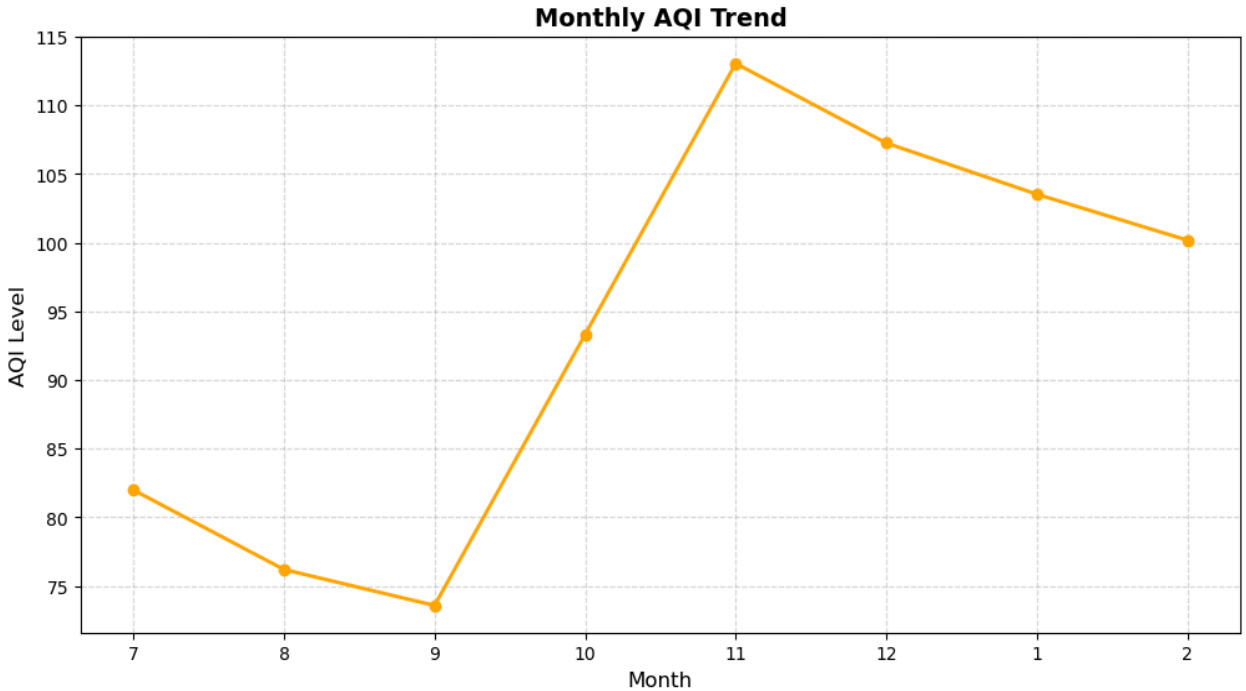
PM2.5 shows a direct high correlation.

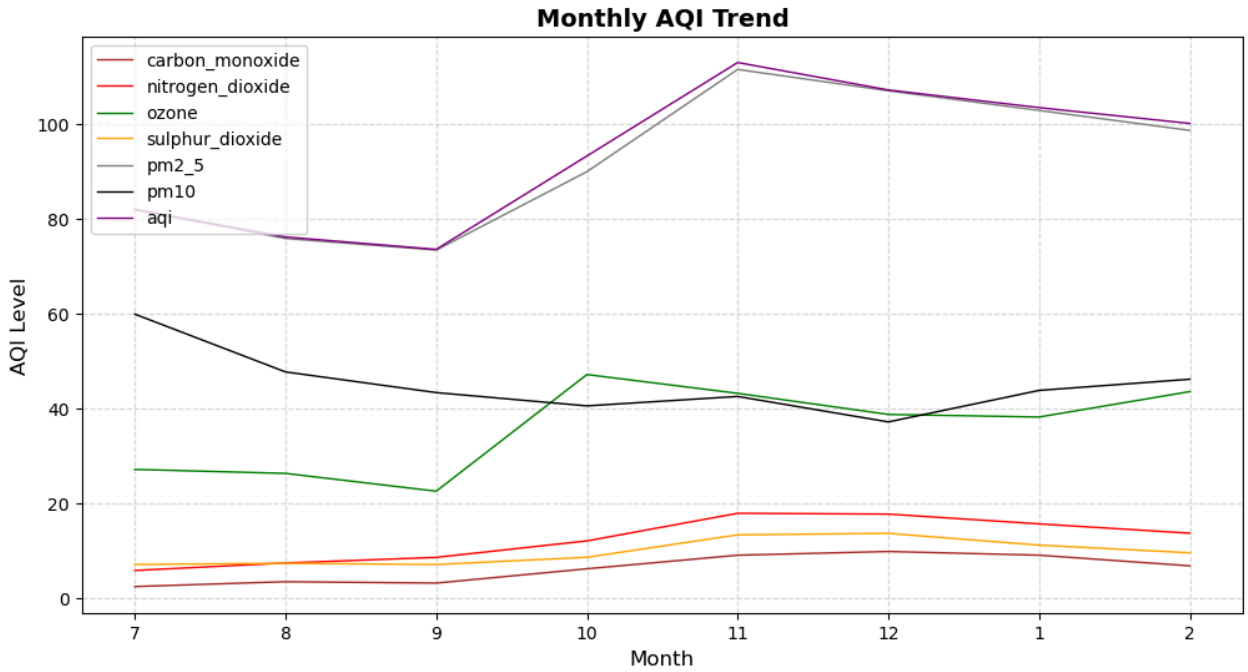
Average Hourly AQI of each day of the week:



Shows higher AQI between 11 and 16 o'clock.

Trend from the past few months:

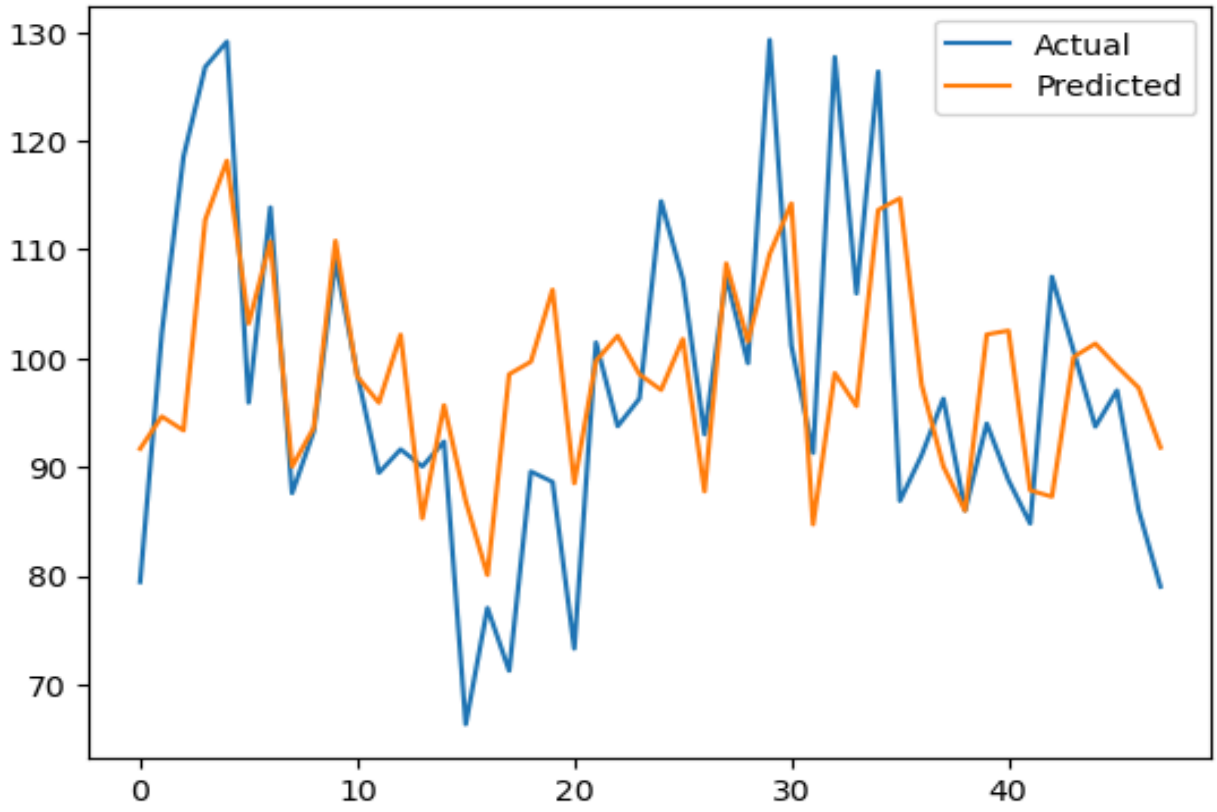




Feature Engineering:

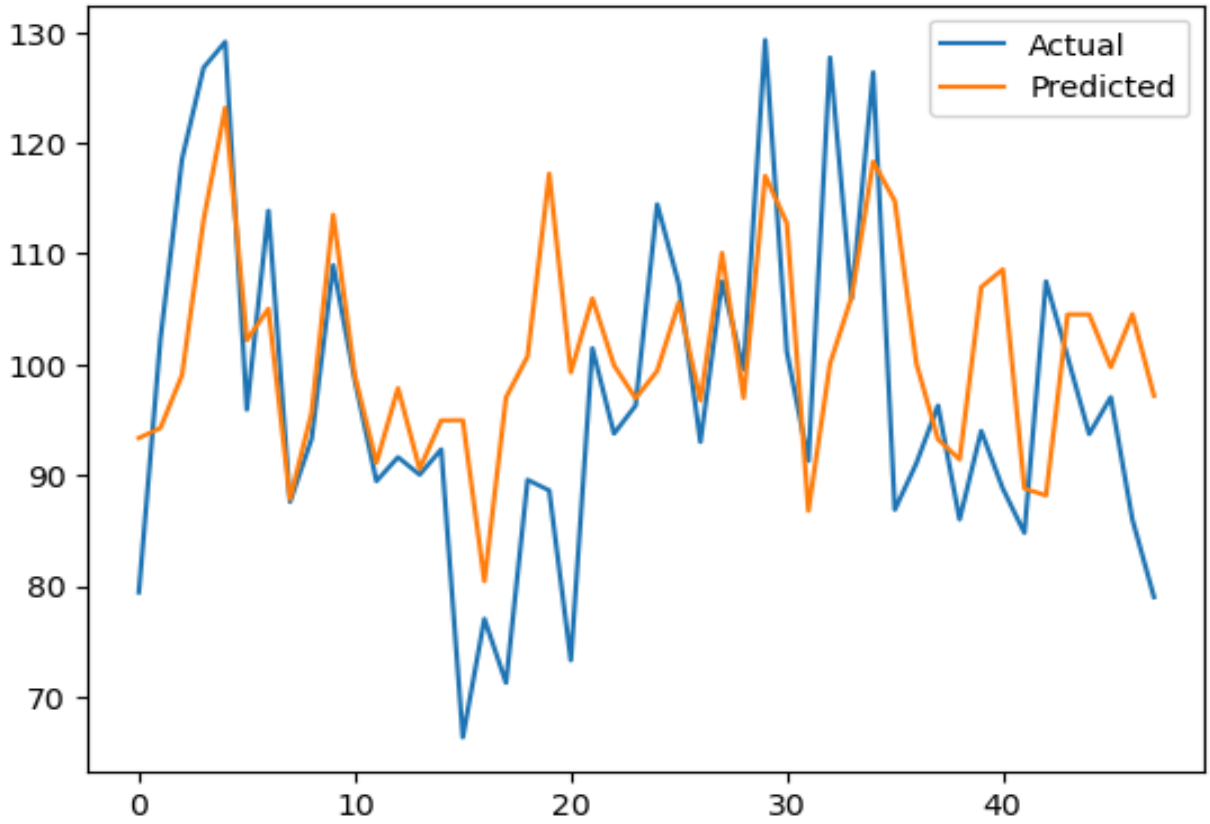
- Converted hourly data to daily.
- Created new datetime columns: year, month, day, day of week, weekend.
- Created lag features

Random Forest performance on test set:



RMSE Score: 12.32
Mean Absolute Error: 9.58
Mean Absolute Percentage Error: 10.09%
R^2 Score: 0.32

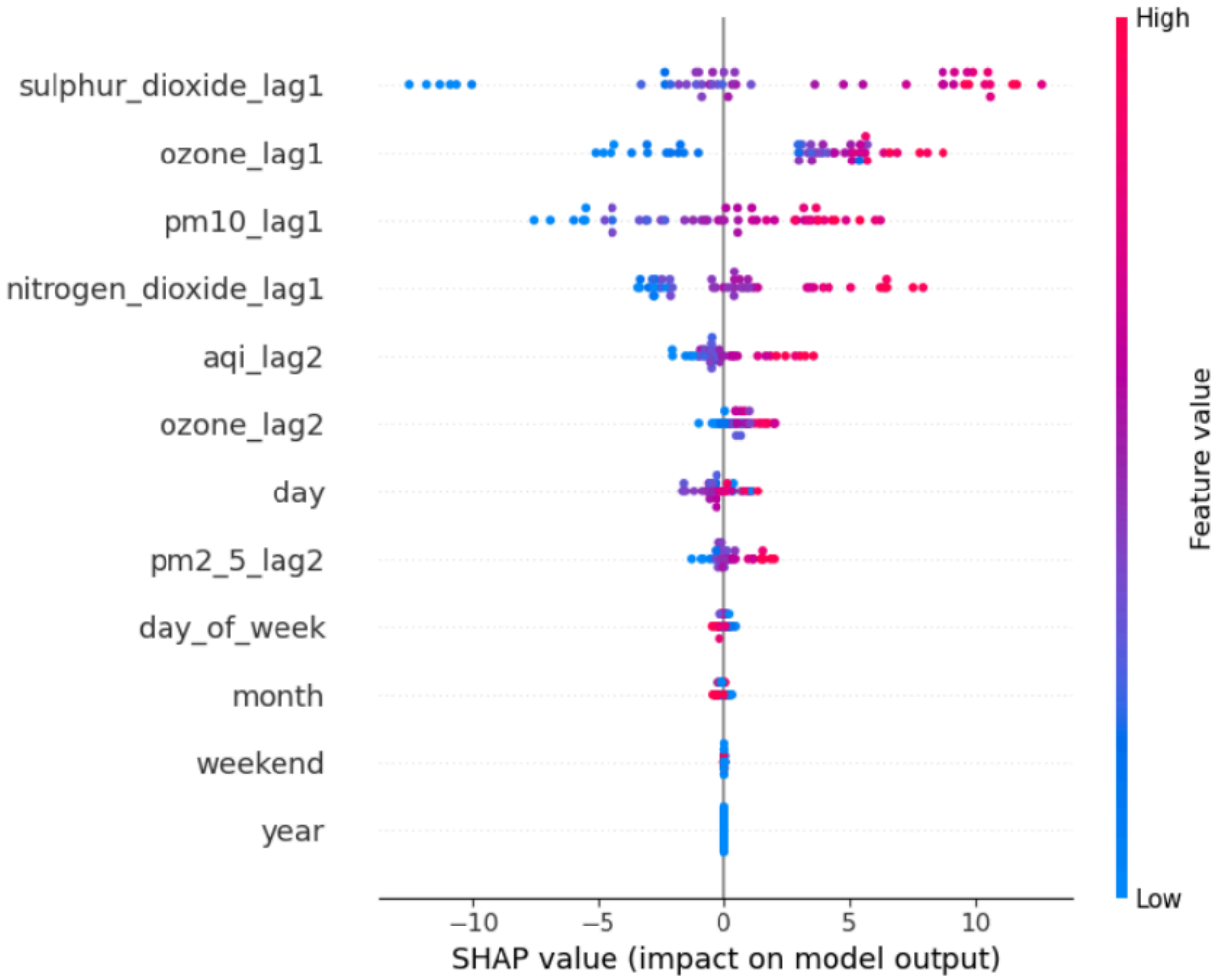
Gradient Boost performance on test set:



RMSE Score: 13.13
Mean Absolute Error: 9.90
Mean Absolute Percentage Error: 10.78%
R^2 Score: 0.23

The data is non-stationary, which is why model performance is not very good.

SHAP Analysis:



These features are finalized after discarding the ones that had no contribution in the performance.

Methodology

A. Backfill Data

- To fetch the last 8 months of raw pollutant data from the Open Meteo API.
- Create the time-based features: day, month, year, day of week, weekend.
- Storing these features in the Hopsworks feature group.
- Runs only once.

B. Hourly Data Ingestion

- To fetch the hourly data of raw pollutant data from the Open Meteo API.
- Create the features according to the gesture group schema.
- Storing these features in the Hopsworks feature group.
- This script runs automatically every hour through GitHub Actions.

C. Daily Training

- Fetching historical features and targets from the feature group for model training.
- Training the selected models: Random Forest Regressor, Gradient Boosting Regressor, Ridge Regressor.
- Evaluating models using metrics: R2, MAE, MAPE, RMSE.
- Storing the models with metrics in the model registry.
- This script runs automatically every day through GitHub Actions.

D. Deployment

- A simple, easy-to-understand dashboard created to display real-time and forecasted data.
- Using Streamlit and a GitHub repository for deployment.
- https://air-quality-predictor-project.streamlit.app/

Challenges Faced

- Feature engineering and model tuning to reduce overfitting.
- Hopsworks response and project creation issue.

Future Improvements

- Optimizing features and models to reduce overfitting and for better performance.
- Implementing automated alerts for hazardous AQI levels to improve user safety.
- Including more features from different sources for better prediction

Conclusion

The AQI Predictor project successfully implemented a 100% serverless end-to-end system to forecast Karachi's air quality for the next three days. By leveraging GitHub Actions for automation, the system maintains a continuous cycle of hourly data ingestion and daily model retraining. Integrating Hopsworks for feature storage and Streamlit for real-time visualization. Despite initial challenges with feature tuning and platform response, the project provides a foundation for future enhancements like multi-source data integration and hazardous level alerts.